



FACULTY OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY
DEPARTMENT OF COMMUNICATION TECHNOLOGY AND NETWORK

GROUP PROJECT

DESIGN AND ANALYSIS OF ALGORITHM (CSC4202)

SEMESTER II 2025/2026

COURSE NAME : DESIGN AND ANALYSIS OF ALGORITHM
COURSE CODE : CSC4202
GROUP : 2
PROJECT TITLE : OPTIMISING FLASH FLOOD EVACUATION ROUTES AND TRAFFIC
BOTTLENECK MITIGATION ON A HIGHWAY NETWORK

LECTURER: DR NUR ARZILAWATI MD YUNUS

NAME	MATRIC NUMBER
1. NURUL WAHIDA BINTI MOHD JISMANI (Leader)	225585
2. KRISSHA REDDY A/P APPLASWAMY REDDY	223180
3. SUHA BINTI IRWAN	223361
4. NURFATHIN AQILAH BINTI RASHIDE	222366
5. LIU MIAOYAN	218479

Table of Contents

1.0 Introduction.....	3
2.0 Development of Model.....	5
3.0 Specification of Algorithm.....	8
3.1 Sorting Algorithms.....	8
3.2 Divide and Conquer (DAC).....	8
3.3 Dynamic Programming (DP).....	8
3.4 Greedy Algorithms.....	8
3.5 Graph Algorithms.....	8
3.6 Selected Algorithm: Dijkstra's Algorithm.....	9
3.7 Algorithm Comparison.....	9
4.0 Designing Algorithm.....	10
4.1 Dijkstra's Algorithm Pseudocode.....	10
4.2 Dijkstra's Algorithm Flowchart.....	11
5.0 Checking Correctness.....	12
5.1 Initialization.....	12
5.2 Maintenance.....	12
5.3 Termination.....	12
6.0 Analysis of Algorithm.....	14
6.1 Time Complexity Analysis.....	14
6.2 Best,Average and Worst Case Analysis.....	14
7.0 Implementation.....	15
7.1 Graph creation.....	15
7.2 Dijkstra Function.....	15
7.3 Flood Avoidance Mechanism.....	16
7.4 Mandatory Residential Stops.....	17
7.5 Multi-Destination Route Computation.....	17
7.6 Output Function.....	17
7.7 Java Source Code.....	18
8.0 Program Testing.....	27
9.0 Conclusion.....	29
10.0 References.....	30
11.0 Appendices.....	31

1.0 Introduction

Flash floods are among the most common natural disasters in Malaysia and frequently occur during periods of heavy rainfall. In highly urbanized areas such as Selangor, flash floods can develop rapidly due to drainage limitations and the large volume of surface runoff. These events often cause severe disruptions to transportation networks by flooding roads, damaging infrastructure, and creating traffic congestion that affects both residents and emergency responders (Department of Irrigation and Drainage Malaysia, 2023).

In this project, a flash flood disaster is assumed to occur within a highway network in Selangor. Several residential areas become affected by rising floodwaters and require immediate evacuation. At the same time, emergency relief trucks carrying food supplies, medical equipment, and rescue personnel must travel from emergency operation centers to designated evacuation centers. The highway network consists of several highway junctions, a highway interchange, and a highway checkpoint that provide multiple possible routes for emergency vehicles. However, some highway segments become inaccessible due to flooding, while others experience heavy traffic congestion caused by evacuation activities. One section of the highway network is severely affected by flooding, making it a critical area that may delay or prevent rescue vehicles from passing through. As a result, determining the most efficient route becomes a significant challenge for emergency management authorities.

The main problem addressed in this project is how to identify the shortest and most efficient route for evacuation and relief operations within the affected highway network. During a disaster, emergency personnel often have multiple routes available, but not all routes are equally safe or efficient. Choosing an unsuitable route may increase travel time, create additional traffic bottlenecks, and delay the delivery of critical assistance to affected communities. Therefore, an effective route-planning mechanism is needed to support decision-making during emergency situations.

Finding the optimal route is important because every minute is critical during a flash flood emergency. Faster evacuation and relief distribution can help reduce the risk to affected residents, improve the effectiveness of rescue operations, and minimize delays caused by traffic congestion (National Disaster Management Agency, 2024). By identifying the most efficient path through the highway network, emergency authorities can allocate resources more effectively and ensure that assistance reaches affected locations as quickly as possible.

The goal of this project is to determine the shortest evacuation and relief route from the emergency operation center to one of the designated evacuation centers while travelling through the required residential areas and avoiding flooded road sections. The route should minimize the total travel cost while avoiding roads that are flooded or heavily congested. To achieve this goal, the highway network is modeled as a weighted directed graph, where locations are represented as nodes and roads are represented as edges with associated travel costs.

The expected output of the proposed system is the optimal route from the source location to the designated evacuation center or affected area. Depending on road accessibility, emergency vehicles may be directed to one of the designated evacuation centers using the most efficient available route. The generated route includes all mandatory residential stops before reaching the selected evacuation center. The system will display the sequence of locations along the selected route together with the total travel distance or travel time. This information can assist emergency authorities in making faster and more accurate decisions during flash flood evacuation and relief operations.

2.0 Development of Model

The flash flood evacuation problem is modeled using a weighted directed graph, where each road segment represents a one-way highway connection. In this model, the vertices (nodes) represent important locations within the evacuation network, such as the emergency operation center, highway junctions, residential areas, a highway interchange, a flood-affected road section, a highway checkpoint, and evacuation centers. The directed edges represent the permitted travel direction between connected highway locations. Each road is assigned a weight that represents the travel distance (in kilometres) between two connected locations.

In the proposed scenario, a rescue truck is dispatched from an emergency operation center to deliver aid and assist evacuation activities in flood-affected areas. Due to flooding conditions, multiple routes may exist between the source and destination locations. The rescue truck is required to travel through the designated residential areas before reaching an evacuation center while avoiding flooded road sections. Therefore, the system must identify the route with the shortest total travel distance.

The objective of the model is to minimize the total travel distance from the source node to the destination node. This objective can be expressed as:

$$\text{Minimize } \sum_{i=1}^n w_i$$

where w_i represents the weight (travel distance) of each road segment included in the selected route.

Several constraints are considered in this model. First, all edge weights must be non-negative because travel distance cannot have negative values. Second, rescue vehicles have limited fuel and resources, requiring the selected route to be as efficient as possible. Third, a valid route can only exist if the roads between the source and destination locations are connected. Roads that are completely flooded are treated as inaccessible during route computation and cannot be selected as part of the evacuation route.

Node	Description
A	Emergency Operation Center
B	Highway Junction 1
C	Highway Junction 2
D	Residential Area 1
E	Residential Area 2

F	Highway Interchange
G	Flood Affected Road Section
H	Highway Checkpoint
I	Residential Area 3
J	Evacuation Center 1
K	Evacuation Center 2

Table 2.1: Node Representation of the Highway Network

Edge	Distance (km)
A - B	5
A - C	8
A - H	12
B - D	6
B - G	7
C - D	4
C - F	9
D - E	5
D - G	3
E - I	6
E - J	8
F - G	4
G - H	5
H - J	4
H - K	9

I - J	3
I - K	7
J - K	5

Table 2.2: Edge Representation and Travel Distance

Using this model, the highway network can be analyzed systematically to determine the shortest evacuation and relief route. The resulting route will support emergency authorities in making faster and more effective decisions during flash flood situations.

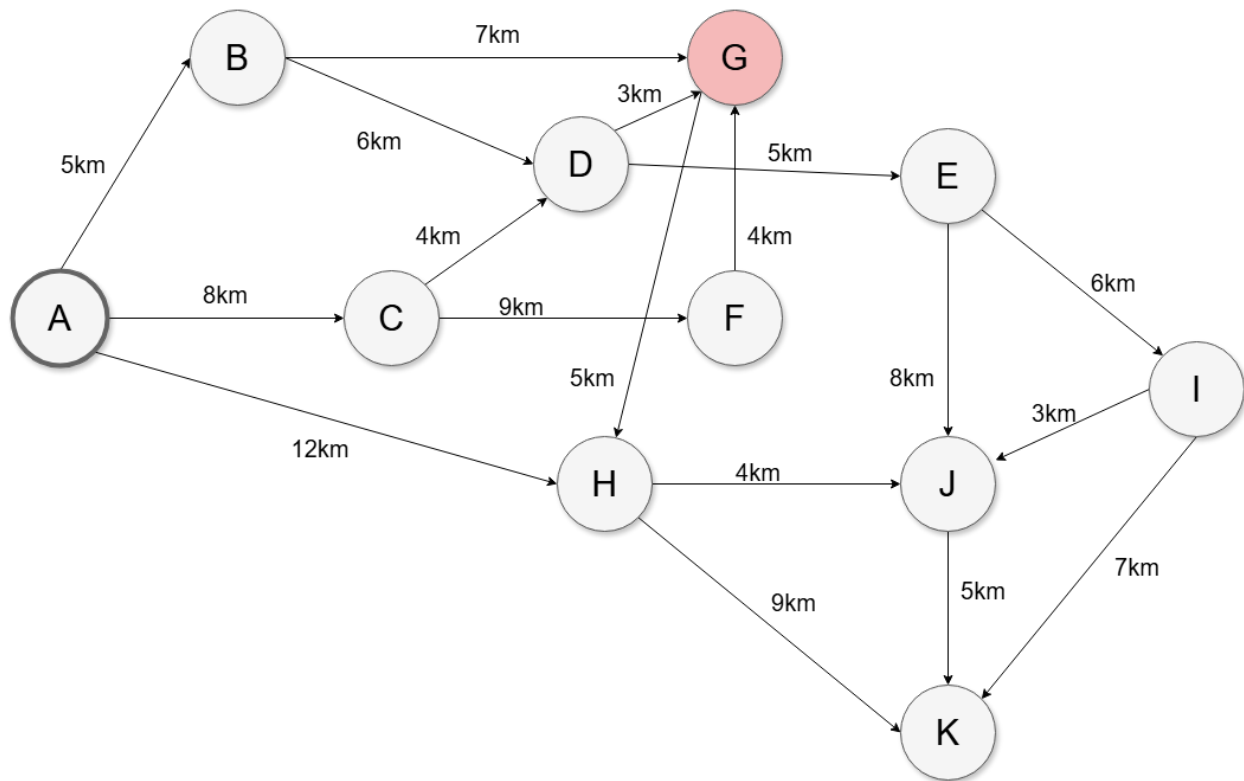


Figure 2.1: Weighted directed graph representation of the proposed highway evacuation network.

3.0 Specification of Algorithm

The flash flood evacuation problem requires finding the most efficient route through a highway network while considering travel distance and road conditions. Several algorithm paradigms were evaluated before selecting the most suitable solution for this project.

3.1 Sorting Algorithms

Sorting algorithms such as Merge Sort and Quick Sort are designed to arrange data in a specific order. Their main strength is the ability to efficiently organize large datasets. According to Mohammadagha (2025), these algorithms also explore improvements such as adaptive methods and parallelisation to enhance performance. However, sorting algorithms cannot directly determine the shortest route between two locations in a transportation network. Therefore, they are not suitable for solving the evacuation routing problem.

3.2 Divide and Conquer (DAC)

Divide and Conquer algorithms solve problems by dividing them into smaller subproblems and combining their solutions. As explained by GeeksforGeeks (2026), this approach is commonly applied in algorithms such as Merge Sort and Binary Search. While this approach is effective for many computational problems, it does not naturally model route optimization within a road network. As a result, its suitability for this project is limited.

3.3 Dynamic Programming (DP)

Dynamic Programming is commonly used for optimization problems that exhibit overlapping subproblems and optimal substructure. Gonzalez and Zhao (2020) explain that this technique improves efficiency by storing previously computed results to avoid redundant calculations. It can produce optimal solutions for route-planning problems. However, implementing Dynamic Programming for a large highway network may require significant memory and a more complex state representation. Therefore, it is not the most efficient choice for the proposed scenario.

3.4 Greedy Algorithms

Greedy algorithms make the best local decision at each step with the goal of achieving a globally optimal solution. . According to García (2025), this method is computationally efficient and easy to implement in many cases. However, not all greedy algorithms guarantee an optimal solution for every optimization problem. Their effectiveness depends on the characteristics of the problem being solved.

3.5 Graph Algorithms

Graph algorithms are specifically designed to solve problems involving networks of interconnected nodes and edges (GeeksforGeeks, 2024). Since the highway system can be

represented as a weighted directed graph, graph algorithms provide a natural and efficient way to model the evacuation problem. They can identify optimal routes between locations while considering travel costs such as distance or travel time.

3.6 Selected Algorithm: Dijkstra's Algorithm

Among the available paradigms, a graph-based approach using Dijkstra's Algorithm was selected for this project. Dijkstra's Algorithm is a shortest-path algorithm that finds the minimum-cost route between a source node and a destination node in a weighted directed graph with non-negative edge weights. The algorithm repeatedly selects the unvisited node with the smallest known distance and updates the distances of its neighboring nodes until the destination is reached.

Dijkstra's Algorithm was chosen because it is well suited to the characteristics of the proposed flash flood evacuation scenario. The highway network can be modeled as a weighted directed graph where locations are represented as nodes and roads are represented as edges. The travel distance or travel time assigned to each road segment serves as the edge weight. Since travel distances and travel times cannot be negative, the assumptions required by Dijkstra's Algorithm are satisfied. Furthermore, the algorithm guarantees the shortest path while maintaining a relatively efficient time complexity, making it suitable for emergency route planning during disaster situations (IBM, 2024).

3.7 Algorithm Comparison

Paradigm	Strength	Weakness	Suitability
Sorting	Efficient data organization	Cannot find routes	Not suitable
Divide and Conquer	Solves smaller subproblems efficiently	Poor route-network representation	Less suitable
Dynamic Programming	Produces optimal solutions	Higher complexity and memory usage	Moderately suitable
Greedy	Fast and simple decisions	May not always guarantee optimum	Suitable
Graph algorithm	Naturally models road networks and shortest paths	Requires graph representation	Most suitable

Table 3.1: Comparison of Algorithm Paradigms

4.0 Designing Algorithm

4.1 Dijkstra's Algorithm Pseudocode

Dijkstra(Graph, Source):

1. Set distance of Source node to 0
2. For each location v in Graph:
 - If v is not Source:
 - Set distance[v] = infinity
 - Set previous[v] = undefined
 - Add v into unvisited list Q
3. While Q is not empty:
 - a. Select the location u with the smallest distance
 - b. Remove u from Q
 - c. For each connected road (neighbor) v of u:
 - Calculate:
 $\text{newDistance} = \text{distance}[u] + \text{roadDistance}(u,v)$
 - If newDistance is smaller than distance[v]:
 - Update distance[v]
 - Set previous[v] = u
4. Return shortest distance and route path

4.2 Dijkstra's Algorithm Flowchart

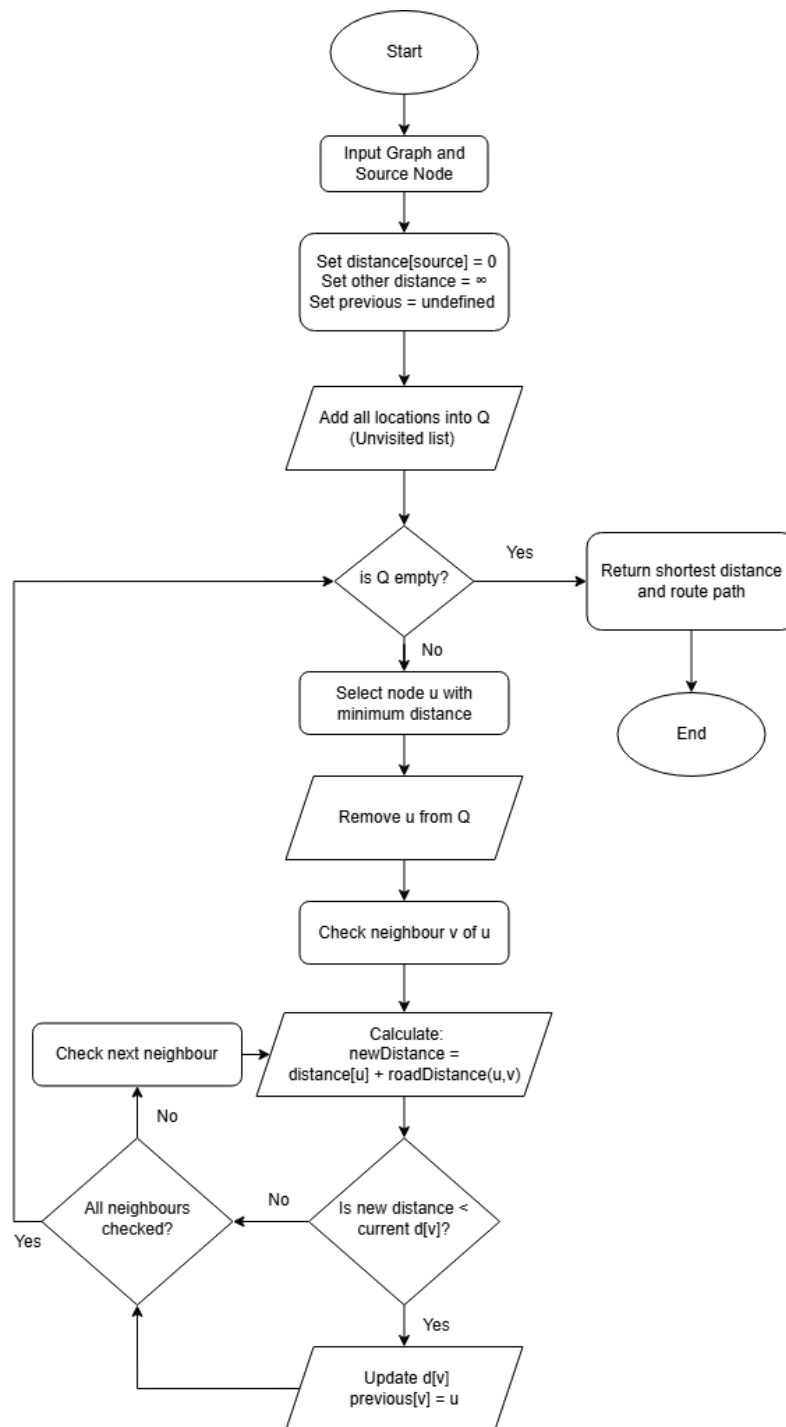


Figure 4.1: Flowchart execution of the rescue truck shortest-path calculation using Dijkstra's Algorithm.

5.0 Checking Correctness

The correctness of Dijkstra's Algorithm is fundamental to its application in finding the shortest paths in a graph with non-negative edge weights. The proof of correctness typically relies on the concept of a loop invariant that holds true throughout the algorithm's execution. This invariant ensures that at each step, the algorithm correctly identifies the shortest path to a subset of vertices.

Let's define the loop invariant for Dijkstra's Algorithm, referring to the pseudocode in Section 4.1:

Loop Invariant: At the beginning of each iteration of the while Q is not empty loop, for every vertex u that has been removed from Q (i.e., added to the set of settled nodes), $\text{distance}[u]$ is the true shortest path distance from the source Source to u .

5.1 Initialization

Before the first iteration of the while loop, the distance of the Source node is set to 0, and all other nodes have their distance set to infinity. The Source node is the first to be selected and removed from Q because its distance is 0, which is the smallest possible distance. At this point, $\text{distance}[\text{Source}]$ is indeed the true shortest path distance from Source to itself. Thus, the loop invariant holds true before the first iteration.

5.2 Maintenance

Assume that the loop invariant holds true at the beginning of an iteration. That is, for all vertices u already removed from Q, $\text{distance}[u]$ represents the true shortest path distance from Source to u . In the current iteration, the algorithm selects a vertex u from Q with the smallest $\text{distance}[u]$ among all vertices currently in Q.

Consider any other vertex v still in Q. If there were a shorter path from Source to u that involved a vertex v' still in Q, then $\text{distance}[v']$ would be less than $\text{distance}[u]$ (since v' would be part of a shorter path to u and u was chosen as the minimum). This contradicts the selection of u as the vertex with the smallest distance from Q. Therefore, when u is selected, $\text{distance}[u]$ must be the true shortest path distance from Source to u .

When u is removed from Q, the algorithm then relaxes its neighbors v . For each neighbor v , it checks if $\text{newDistance} = \text{distance}[u] + \text{roadDistance}(u,v)$ is less than $\text{distance}[v]$. If it is, $\text{distance}[v]$ is updated. This relaxation step is crucial. It ensures that $\text{distance}[v]$ always stores the shortest path found so far from Source to v through already settled nodes or nodes currently in Q. Because u 's distance is now finalized as the shortest path, any path through u to v is correctly evaluated.

5.3 Termination

The while loop continues until Q is empty. When Q is empty, all reachable vertices have been removed from Q, and their distance values have been finalized. According to the loop invariant,

at the moment each vertex u is removed from Q , $\text{distance}[u]$ is the true shortest path distance from Source to u . Therefore, upon termination, the algorithm has successfully found the shortest path from the Source to all other reachable vertices in the graph.

This proof relies on the critical assumption that all edge weights are non-negative, which is satisfied in the flash flood evacuation scenario where `roadDistance` represents physical distances or travel times. If negative edge weights were present, Dijkstra's Algorithm would not guarantee correctness, and other algorithms like Bellman-Ford would be required.

6.0 Analysis of Algorithm

The implemented Dijkstra's Algorithm uses an adjacency list and a priority queue (min-heap) to efficiently determine the shortest route within the highway network. Let V represent the number of nodes (locations) and E represent the number of edges (roads). The use of a priority queue enables efficient selection of the node with the minimum tentative distance during the execution of the algorithm.

6.1 Time Complexity Analysis

The overall time complexity of Dijkstra's Algorithm implemented using an adjacency list and a priority queue is:

$$O((V+E)\log V)$$

where:

- V represents the number of locations in the network
- E represents the number of roads connecting the locations

6.2 Best,Average and Worst Case Analysis

Case	Time Complexity	Description
Best Case	$O(V \log V)$	The destination node is reached early, allowing the algorithm to terminate without exploring the remaining nodes in the graph.
Average Case	$O((V+E) \log V)$	A typical evacuation scenario where several alternative routes are evaluated before finding the shortest path.
Worst Case	$O((V+E) \log V)$	The algorithm explores all reachable nodes and edges before determining the shortest path.

Table 6.1: Asymptotic Performance of Dijkstra's Algorithm.

7.0 Implementation

The proposed system is implemented by using a java programming language. The highway network is represented by a weighted directed graph where each of the locations is represented as a vertex and each of the road connections is represented as an edge with each own distance value. The architecture was compiled and executed using an array-backed Priority Queue implementation in Java to ensure an $O((E + V) \log V)$ time complexity. Upon execution, the system successfully avoided the flooded road section (Node G) by computing alternative routes while satisfying all mandatory residential stops.

7.1 Graph creation

The highway network is represented as an adjacency list using Java's `HashMap<Character, List<Edge>>`. Each edge stores the destination node and travel distance. This representation enables efficient traversal of neighbouring nodes during Dijkstra's Algorithm. Each of the roads is represented by its travel distance.

```
Map<Character, List<Edge>> graph = new HashMap<>();  
  
addEdge(graph, 'A', 'B', 5);  
addEdge(graph, 'A', 'C', 8);  
addEdge(graph, 'B', 'D', 6);
```

Figure 7.1: Snippet code of graph construction

Example:

From location	To location	Distance (km)
Emergency centre	Village A	5
Emergency centre	Village B	3
Village B	Village A	2

7.2 Dijkstra Function

The Dijkstra function is responsible for finding the shortest route available. The function performs the following steps:

1. Set the source distance as 0.
2. Set all other distances as infinity.
3. Select the nearest unvisited location.

4. Update the distance function of the connected locations.
5. Repeat until all the locations are visited and processed.

The algorithm will calculate the minimum distance required for the rescue vehicles to travel from the relief centre to the affected areas. The use of a priority queue improves computational efficiency by avoiding repeated scanning of all vertices. Consequently, the overall time complexity of the implementation becomes:

$$O((V+E)\log V)$$

where:

- V represents the number of vertices.
- E represents the number of edges.

Example implementation:

```
PriorityQueue<Node> pq = new PriorityQueue<>();  
  
distances.put(start, 0);  
pq.add(new Node(start, 0));
```

Figure 7.2 : Snippet code of Dijkstra function

7.3 Flood Avoidance Mechanism

To simulate real-world flash flood conditions, the flood-affected section represented by node G was treated as inaccessible during route computation. Whenever Dijkstra's algorithm encounters node G, the node is ignored and excluded from further exploration. This approach ensures that all generated routes avoid flooded areas and remain operational for emergency response vehicles.

Implementation snippet:

```
if (v == 'G') {  
    continue;  
}
```

Figure 7.3: Node G as flooded highway section and inaccessible during route computation

7.4 Mandatory Residential Stops

Unlike conventional shortest path problems that compute a route directly from a source node to a destination node, this project introduces mandatory residential checkpoints that must be visited before reaching an evacuation center. To achieve this, the implementation utilizes a checkpoint array which is `char[] checkpoints = {'A', 'D', 'E', 'I', dest};`

The program then repeatedly executes Dijkstra's algorithm between consecutive checkpoints:

```
for (int i = 0; i < checkpoints.length - 1; i++) {  
    runDijkstra(graph,  
                checkpoints[i],  
                checkpoints[i + 1]);  
}
```

Figure 7.4: Looping Dijkstra's Algorithm

7.5 Multi-Destination Route Computation

The implementation supports multiple evacuation centers by treating the destination node as a variable parameter rather than a fixed value. In this project, two evacuation centers were considered which are Evacuation Center 1 (J) and Evacuation Center 2 (K). The program computes the shortest feasible route to each evacuation center independently after all mandatory residential areas have been visited.

```
char[] finalDestinations = {'J', 'K'};
```

Figure 7.5: Snippet code of final destination node

The use of multiple evacuation centers increases operational flexibility and allows emergency management authorities to maintain alternative evacuation options if one center becomes unavailable during disaster conditions.

7.6 Output Function

After all route segments have been computed, the individual paths are combined to generate a complete evacuation itinerary. The total travel distance is obtained by summing the distances of each segment. The output function will display the final shortest destination distance after the algorithm is completed.

The program will display:

- Final evacuation centre

- Complete evacuation route
- Total travel distance
- Route validity (if no feasible route exists)

This allows the emergency responder to identify which path is the fastest route for efficient evacuation and relief distribution.

7.7 Java Source Code

```
import java.util.*;

public class FloodEvacuationMultiStopRouter {

    static class Edge {

        char target;

        int weight;

        Edge(char target, int weight) {

            this.target = target;

            this.weight = weight;

        }

    }

    static class Node implements Comparable<Node> {

        char name;

        int distance;

        Node(char name, int distance) {
```

```

        this.name = name;

        this.distance = distance;
    }

    @Override

    public int compareTo(Node other) {

        return Integer.compare(this.distance, other.distance);
    }
}

public static void main(String[] args) {

    Map<Character, List<Edge>> graph = new HashMap<>();

    char[] nodes = {'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K'};

    for (char node : nodes) {

        graph.put(node, new ArrayList<>());
    }

    // Graph edges mapping (Strictly One-Way / Directed Graph)

    addEdge(graph, 'A', 'B', 5);

    addEdge(graph, 'A', 'C', 8);

    addEdge(graph, 'A', 'H', 12);

    addEdge(graph, 'B', 'G', 7);

    addEdge(graph, 'B', 'D', 6);

    addEdge(graph, 'C', 'D', 4);

    addEdge(graph, 'C', 'F', 9);

```

```
addEdge(graph, 'D', 'G', 3);

addEdge(graph, 'D', 'E', 5);

addEdge(graph, 'F', 'G', 4);

addEdge(graph, 'G', 'H', 5);

addEdge(graph, 'H', 'J', 4);

addEdge(graph, 'H', 'K', 9);

addEdge(graph, 'E', 'J', 8);

addEdge(graph, 'E', 'I', 6);

addEdge(graph, 'I', 'J', 3);

addEdge(graph, 'I', 'K', 7);

addEdge(graph, 'J', 'K', 5);


// Map node characters to full descriptions for output readability
Map<Character, String> names = new HashMap<>();

names.put('A', "Emergency Operation Center");

names.put('B', "Highway Junction 1");

names.put('C', "Highway Junction 2");

names.put('D', "Residential Area 1");

names.put('E', "Residential Area 2");

names.put('F', "Highway Interchange");

names.put('G', "Flood Affected Road Section");

names.put('H', "Highway Checkpoint");

names.put('I', "Residential Area 3");

names.put('J', "Evacuation Center 1");
```

```

names.put('K', "Evacuation Center 2");

System.out.println("=====
=====");

    System.out.println(" MANDATORY RESIDENTIAL SUPPLY ROUTE (BYPASSING FLOOD
ZONE G)");

    System.out.println(" Required Stops: Residential Area 1 (D) -> 2 (E) -> 3 (I)");

System.out.println("=====
=====\\n");

// Define the final endpoints we want to test after clearing all residential zones
char[] finalDestinations = {'J', 'K'};

for (char dest : finalDestinations) {

    StringBuilder fullPath = new StringBuilder();

    int totalDistance = 0;

    boolean routeValid = true;

    // Sequential segments: A -> D -> E -> I -> Destination
    char[] checkpoints = {'A', 'D', 'E', 'I', dest};

    for (int i = 0; i < checkpoints.length - 1; i++) {

        DijkstraResult result = runDijkstra(graph, checkpoints[i], checkpoints[i + 1]);

```

```

        if (result.distance == Integer.MAX_VALUE) {

            routeValid = false;

            break;

        }

        totalDistance += result.distance;

        // Append path segments seamlessly

        if (i == 0) {

            fullPath.append(result.path);

        } else {

            // Avoid duplicating the start node of the next leg (skips the first 5 characters "X -> ")

            fullPath.append(" -> ").append(result.path.substring(5));

        }

    }

    System.out.println("Final Destination: " + names.get(dest) + " (" + dest + ")");

    if (routeValid) {

        System.out.println("  Total Multi-Stop Distance: " + totalDistance + " km");

        System.out.println("  Complete Itinerary      : " + fullPath.toString());

    } else {

        System.out.println("  Route status: NO VIABLE ROUTE satisfying all mandatory
stops.");

    }

```

```

        System.out.println("-----");
    }
}

private static void addEdge(Map<Character, List<Edge>> graph, char source, char target, int
weight) {
    graph.get(source).add(new Edge(target, weight));
}

// Helper class to store individual segment outcomes
static class DijkstraResult {
    int distance;
    String path;

    DijkstraResult(int distance, String path) {
        this.distance = distance;
        this.path = path;
    }
}

// Dijkstra execution targeting a specific single destination node, ignoring flooded nodes
private static DijkstraResult runDijkstra(Map<Character, List<Edge>> graph, char start, char
end) {
    Map<Character, Integer> distances = new HashMap<>();
    Map<Character, Character> predecessors = new HashMap<>();

```

```

PriorityQueue<Node> pq = new PriorityQueue<>();

for (char node : graph.keySet()) {
    distances.put(node, Integer.MAX_VALUE);
}

distances.put(start, 0);
pq.add(new Node(start, 0));

while (!pq.isEmpty()) {
    Node current = pq.poll();
    char u = current.name;

    if (current.distance > distances.get(u)) continue;
    if (u == end) break; // Optimization: destination reached

    for (Edge edge : graph.get(u)) {
        char v = edge.target;

        // FLOOD SAFETY CHECK: Skip node G entirely
        if (v == 'G') {
            continue;
        }

        int newDist = distances.get(u) + edge.weight;
    }
}

```



```

        if (newDist < distances.get(v)) {
            distances.put(v, newDist);
            predecessors.put(v, u);
            pq.add(new Node(v, newDist));
        }
    }
}

int targetDist = distances.get(end);
if (targetDist == Integer.MAX_VALUE) {
    return new DijkstraResult(Integer.MAX_VALUE, "");
}

// Reconstruct path string
List<Character> pathList = new ArrayList<>();
char curr = end;
while (predecessors.containsKey(curr)) {
    pathList.add(curr);
    curr = predecessors.get(curr);
}
pathList.add(curr);
Collections.reverse(pathList);

```

```
StringBuilder sb = new StringBuilder();

for (int i = 0; i < pathList.size(); i++) {

    sb.append(pathList.get(i));

    if (i < pathList.size() - 1) sb.append(" -> ");

}

return new DijkstraResult(targetDist, sb.toString());

}

}
```

8.0 Program Testing

To verify the functional correctness and accuracy of the implemented shortest-path routing application, a controlled test execution was conducted. The network's layout structure and connection weights can be mathematically represented in the system using an adjacency matrix.

Location	Node A (km)	Node B (km)	Node C (km)	Node D (km)	Node E (km)	Node F (km)	Node G (km)	Node H (km)	Node I (km)	Node J (km)	Node K (km)
Node A (Emergency Center)	0	5	8	0	0	0	0	12	0	0	0
Node B (Highway Junction 1)	5	0	0	6	0	0	7	0	0	0	0
Node C (Highway Junction 2)	8	0	0	4	0	0	0	0	0	0	0
Node D (Residential Area 1)	0	6	4	0	5	0	3	0	0	0	0
Node E (Residential Area 2)	0	0	0	5	0	0	0	0	6	8	0
Node F (Highway Interchange)	0	0	9	0	0	0	4	0	0	0	0
Node G (Flood Affected Road Section)	0	7	0	3	0	4	0	5	0	0	0
Node H (Highway Checkpoint)	12	0	0	0	0	0	5	0	0	4	9
Node I (Residential Area 3)	0	0	0	0	6	0	0	0	0	3	7
Node J (	0	0	0	0	8	0	0	4	3	0	5

Evacuation Center 1)											
Node K (Evacuation Center 2)	0	0	0	0	0	0	9	7	5	0	0

Table 8.1: Adjacency Matrix Data Input Representation for the Highway Network

```

=====
MANDATORY RESIDENTIAL SUPPLY ROUTE (BYPASSING FLOOD ZONE G)
Required Stops: Residential Area 1 (D) -> 2 (E) -> 3 (I)
=====

Final Destination: Evacuation Center 1 (J)
Total Multi-Stop Distance: 25 km
Complete Itinerary      : A -> B -> D -> E -> I -> J
-----

Final Destination: Evacuation Center 2 (K)
Total Multi-Stop Distance: 29 km
Complete Itinerary      : A -> B -> D -> E -> I -> K
-----

```

Figure 8.1: Screenshot of the finalised Java application execution terminal output.

Figure 8.1 shows that the program successfully generated evacuation routes from the Emergency Operation Center (A) to both Evacuation Center 1 (J) and Evacuation Center 2 (K) while satisfying all operational constraints. The algorithm enforced mandatory visits to Residential Area 1 (D), Residential Area 2 (E), and Residential Area 3 (I) before proceeding to the final destination. For Evacuation Center 1, the computed route was A → B → D → E → I → J with a total travel distance of 25 km, while the route to Evacuation Center 2 was A → B → D → E → I → K with a total distance of 29 km. Both routes successfully bypassed the flood-affected highway section represented by node G, demonstrating that the flood avoidance mechanism functioned correctly by excluding unsafe paths from consideration. The results indicate that the proposed implementation is capable of generating safe and feasible multi-stop evacuation routes to multiple evacuation centers while adhering to the constraints of mandatory residential supply delivery and flood avoidance.

9.0 Conclusion

This project successfully modeled the flash flood evacuation problem within a highway network as a weighted directed graph and applied the Dijkstra's Algorithm to determine the shortest and safest route from the Emergency Operation Center to the designated evacuation centers. By presenting each highway location as nodes and road segments as weighted edges, the system was able to compute an optimal route that satisfies the mandatory residential stop requirement for all the neighbourhood while avoiding the flood affected road sections that are labeled as Node G in the network graph.

The implementation of Dijkstra's Algorithm in solving these problems is developed in Java using an adjacency list and priority queue. The project is able to achieve the algorithm's theoretical time complexity of $O((V+E) \log V)$ which confirms that the Dijkstra's Algorithm is computationally efficient when extended to handle multiple nodes. The testing of the code showed that the system manage to correctly identified a 25 km route to Evacuation 1 (Node J) and a 29 km route to Evacuation 2 (Node K) while routing through Residential Areas 1, 2, and 3 (Node D, E, I). The code also avoids the flood segment node completely. The correctness of the algorithm was further validated through loop invariant proof method that confirmed that each chosen node's distance value is representing the true shortest path from the source.

To conclude, the project demonstrated that Dijkstra's Algorithm is well suited for time critical disaster response applications such as flash flood evacuation. The model is able to provide emergency response with a practical decision support tool that could respond quickly and identify the most efficient evacuation and relief route. It also helps to reduce the response time and improve the outcome during an emergency situation. Future improvements that could be made include incorporating real-time traffic data with any dynamic edge weight changes to reflect the unpredictable flood conditions.

10.0 References

- Bogdan, P. (2015). Dijkstra algorithm in parallel—Case study. In Proceedings of the 2015 16th International Carpathian Control Conference (ICCC) (pp. 50–53). IEEE. <https://doi.org/10.1109/CarpathianCC.2015.7145044>
- Department of Irrigation and Drainage Malaysia. (2023). *Flood management and mitigation in Malaysia*. Ministry of Natural Resources, Environment and Climate Change. <https://www.water.gov.my>
- García, A. (2025). Greedy algorithms: a review and open problems. *Journal of Inequalities and Applications*, 2025(1). <https://doi.org/10.1186/s13660-025-03254-1>
- GeeksforGeeks. (2026, January 19). *Introduction to Divide and Conquer Algorithm*. GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/introduction-to-divide-and-conquer-algorithm/>
- GeeksforGeeks. (2024, January 17). *Graph Algorithms*. GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/graph-data-structure-and-algorithms/>
- Gonzalez, L., & Zhao, A. (2020). *Dynamic Programming and Applications*. https://math.mit.edu/research/highschool/primes/circle/documents/2020/GonzalezGauss_Zhao_2020.pdf
- IBM. (2024). *What is Dijkstra's algorithm?* <https://www.ibm.com/topics/dijkstra-algorithm>
- International Federation of Red Cross and Red Crescent Societies. (2024). *Floods and flash floods*. <https://www.ifrc.org>
- Mohammadagha, M. (2025). *Hybridization and Optimization Modeling, Analysis, and Comparative Study of Sorting Algorithms: Adaptive Techniques, Parallelization, for Mergesort, Heapsort, Quicksort, Insertion Sort, Selection Sort, and Bubble Sort*. Engrxiv.org. <https://engrxiv.org/preprint/view/4537/7883>
- National Disaster Management Agency. (2024). *Disaster preparedness and emergency response*. Prime Minister's Department. <https://www.nadma.gov.my>

11.0 Appendices

Initial Project Plan

Group Name	Group 5		
Members			
	Name	Email	Phone number
	NURUL WAHIDA BINTI MOHD JISMANI	225585@student.upm.edu.my	011-40262899
	SUHA BINTI IRWAN	223361@student.upm.edu.my	011-26183712
	NURFATHIN AQILAH BINTI RASHIDE	222366@student.upm.edu.my	011-56516323
	LIU MIAOYAN	218479@student.upm.edu.my	0 14-2510726
	KRISSHA REDDY A/P APPLASWAMY REDDY	223180@student.upm.edu.my	016-2676282
Problem scenario description	A flash flood occurs in the Selangor highway network, affecting several residential areas and disrupting road accessibility. Emergency rescue vehicles must determine the shortest and safest evacuation route to deliver aid and transport residents to evacuation centres while avoiding flooded roads.		
Why it is important	Rapid route planning during flash floods helps reduce evacuation time, improves emergency response, and ensures that relief supplies reach affected communities as quickly as possible.		
Problem specification	• Model the highway network as a weighted directed graph. • Represent locations as nodes and roads as edges. • Use travel distance as the edge weight. • Determine the shortest evacuation route while avoiding flooded road sections.		
Potential solutions	• Sorting Algorithms (not suitable) • Divide and Conquer (less suitable) • Dynamic Programming (moderately suitable) • Greedy Algorithm • Graph Algorithm (Dijkstra's Algorithm) — selected solution		

Sketch (framework, flow, interface)	Emergency Operation Center → Highway Network (Graph) → Dijkstra's Algorithm → Shortest Safe Route → Evacuation Center
--	--

Project Proposal Refinement

Group Name	Group 5													
Members	<table><tr><th>Name</th><th>Role</th></tr><tr><td>NURUL WAHIDA BINTI MOHD JISMANI</td><td>Project leader</td></tr><tr><td>SUHA BINTI IRWAN</td><td>Graph designer</td></tr><tr><td>NURFATHIN AQILAH BINTI RASHIDE</td><td>Algorithm developer</td></tr><tr><td>LIU MIAOYAN</td><td>Tester</td></tr><tr><td>KRISSHA REDDY A/P APPLASWAMY REDDY</td><td>Documentation and presentation</td></tr></table>		Name	Role	NURUL WAHIDA BINTI MOHD JISMANI	Project leader	SUHA BINTI IRWAN	Graph designer	NURFATHIN AQILAH BINTI RASHIDE	Algorithm developer	LIU MIAOYAN	Tester	KRISSHA REDDY A/P APPLASWAMY REDDY	Documentation and presentation
	Name	Role												
	NURUL WAHIDA BINTI MOHD JISMANI	Project leader												
	SUHA BINTI IRWAN	Graph designer												
	NURFATHIN AQILAH BINTI RASHIDE	Algorithm developer												
	LIU MIAOYAN	Tester												
	KRISSHA REDDY A/P APPLASWAMY REDDY	Documentation and presentation												
Problem statement	Determine the shortest and safest evacuation route for emergency vehicles during a flash flood in Selangor while avoiding flooded road sections and passing through the required residential areas.													
Objectives	- Model the highway network as a weighted directed graph. - Implement Dijkstra's Algorithm in Java. - Determine the shortest evacuation route. - Evaluate the algorithm's correctness and time complexity.													
Expected output	Display the optimal evacuation route, total travel distance, and selected evacuation center.													
Problem scenario description	A rescue vehicle starts from the Emergency Operation Center and travels through designated residential areas before reaching the most suitable evacuation center while avoiding flooded roads.													
Why it is important	Efficient evacuation routing reduces response time, improves rescue operations, and supports disaster management.													
Problem specification	Weighted directed graph													

	• Non-negative edge weights • Flooded road sections treated as inaccessible • Mandatory residential stops												
Potential solutions	Comparison of Sorting, Divide and Conquer, Dynamic Programming, Greedy, and Graph algorithms. Dijkstra's Algorithm was selected because it efficiently solves the shortest path problem.												
Sketch (framework, flow, interface)	Graph diagram of nodes A–K connected by weighted edges, showing Emergency Operation Center, Residential Areas, Flooded Road Section, Highway Checkpoint, and Evacuation Centers.												
Methodology	<table border="1"> <thead> <tr> <th>Milestone</th><th>Time</th></tr> </thead> <tbody> <tr> <td>Scenario selection and problem analysis</td><td>wk10</td></tr> <tr> <td>Graph modelling and algorithm comparison</td><td>wk11</td></tr> <tr> <td>Java implementation and debugging</td><td>wk12</td></tr> <tr> <td>Algorithm correctness and complexity analysis</td><td>wk13</td></tr> <tr> <td>Portfolio, documentation and presentation preparation</td><td>wk14</td></tr> </tbody> </table>	Milestone	Time	Scenario selection and problem analysis	wk10	Graph modelling and algorithm comparison	wk11	Java implementation and debugging	wk12	Algorithm correctness and complexity analysis	wk13	Portfolio, documentation and presentation preparation	wk14
Milestone	Time												
Scenario selection and problem analysis	wk10												
Graph modelling and algorithm comparison	wk11												
Java implementation and debugging	wk12												
Algorithm correctness and complexity analysis	wk13												
Portfolio, documentation and presentation preparation	wk14												

Project Progress (Week 10 – Week 14)

Milestone 1													
Date (week)	Week 10												
Description/ sketch	• Finalizing the flash flood evacuation scenario. • Designing the highway graph structure. • Identifying the nodes and edges. • Refining the project objectives.												
Role	<table border="1"> <thead> <tr> <th>Name</th><th>Role</th></tr> </thead> <tbody> <tr> <td>NURUL WAHIDA BINTI MOHD JISMANI</td><td>Project leader</td></tr> <tr> <td>SUHA BINTI IRWAN</td><td>Graph designer</td></tr> <tr> <td>NURFATHIN AQILAH BINTI RASHIDE</td><td>Algorithm developer</td></tr> <tr> <td>LIU MIAOYAN</td><td>Tester</td></tr> <tr> <td>KRISSHA REDDY A/P APPLASWAMY REDDY</td><td>Documentation and presentation</td></tr> </tbody> </table>	Name	Role	NURUL WAHIDA BINTI MOHD JISMANI	Project leader	SUHA BINTI IRWAN	Graph designer	NURFATHIN AQILAH BINTI RASHIDE	Algorithm developer	LIU MIAOYAN	Tester	KRISSHA REDDY A/P APPLASWAMY REDDY	Documentation and presentation
Name	Role												
NURUL WAHIDA BINTI MOHD JISMANI	Project leader												
SUHA BINTI IRWAN	Graph designer												
NURFATHIN AQILAH BINTI RASHIDE	Algorithm developer												
LIU MIAOYAN	Tester												
KRISSHA REDDY A/P APPLASWAMY REDDY	Documentation and presentation												

Milestone 2	
Date (Wk)	Week 11

Description/ sketch	- Finding an example of solutions and suitable algorithms to be used in the experiment. - Discussing why the selected Dijkstra's Algorithm is suitable and how it can relate to the shortest path evacuation routing problems.												
Role	<table border="1"> <thead> <tr> <th>Name</th><th>Role</th></tr> </thead> <tbody> <tr> <td>NURUL WAHIDA BINTI MOHD JISMANI</td><td>Project leader</td></tr> <tr> <td>SUHA BINTI IRWAN</td><td>Graph designer</td></tr> <tr> <td>NURFATHIN AQILAH BINTI RASHIDE</td><td>Algorithm developer</td></tr> <tr> <td>LIU MIAOYAN</td><td>Tester</td></tr> <tr> <td>KRISSHA REDDY A/P APPLASWAMY REDDY</td><td>Documentation and presentation</td></tr> </tbody> </table>	Name	Role	NURUL WAHIDA BINTI MOHD JISMANI	Project leader	SUHA BINTI IRWAN	Graph designer	NURFATHIN AQILAH BINTI RASHIDE	Algorithm developer	LIU MIAOYAN	Tester	KRISSHA REDDY A/P APPLASWAMY REDDY	Documentation and presentation
Name	Role												
NURUL WAHIDA BINTI MOHD JISMANI	Project leader												
SUHA BINTI IRWAN	Graph designer												
NURFATHIN AQILAH BINTI RASHIDE	Algorithm developer												
LIU MIAOYAN	Tester												
KRISSHA REDDY A/P APPLASWAMY REDDY	Documentation and presentation												

Milestone 3	
Date (Wk)	Week 12
Description/ sketch	- Implementing Dijkstra's Algorithm into coding. - Debugging any error inside the code. - Discussing the output outcome.

Role		
	Name	Role
	NURUL WAHIDA BINTI MOHD JISMANI	Project leader
	SUHA BINTI IRWAN	Graph designer
	NURFATHIN AQILAH BINTI RASHIDE	Algorithm developer
	LIU MIAOYAN	Tester
	KRISSHA REDDY A/P APPLASWAMY REDDY	Documentation and presentation

Milestone 4		
Date (Wk)	Week 13	
Description/ sketch	• Conducting analysis of correctness and time complexity. • Test different routes and evaluate the algorithm performance.	
Role		
	Name	Role
	NURUL WAHIDA BINTI MOHD JISMANI	Project leader
	SUHA BINTI IRWAN	Graph designer
	NURFATHIN AQILAH BINTI RASHIDE	Algorithm developer
	LIU MIAOYAN	Tester
	KRISSHA REDDY A/P APPLASWAMY REDDY	Documentation and presentation

--	--

Milestone 5													
Date (Wk)	Week 14												
Description/ sketch	• Preparing online portfolio and presentation. • Finalizing the documentation, slides, project explanation, and presentation materials.												
Role	<table border="1"> <thead> <tr> <th>Name</th><th>Role</th></tr> </thead> <tbody> <tr> <td>NURUL WAHIDA BINTI MOHD JISMANI</td><td>Project leader</td></tr> <tr> <td>SUHA BINTI IRWAN</td><td>Graph designer</td></tr> <tr> <td>NURFATHIN AQILAH BINTI RASHIDE</td><td>Algorithm developer</td></tr> <tr> <td>LIU MIAOYAN</td><td>Tester</td></tr> <tr> <td>KRISSHA REDDY A/P APPLASWAMY REDDY</td><td>Documentation and presentation</td></tr> </tbody> </table>	Name	Role	NURUL WAHIDA BINTI MOHD JISMANI	Project leader	SUHA BINTI IRWAN	Graph designer	NURFATHIN AQILAH BINTI RASHIDE	Algorithm developer	LIU MIAOYAN	Tester	KRISSHA REDDY A/P APPLASWAMY REDDY	Documentation and presentation
Name	Role												
NURUL WAHIDA BINTI MOHD JISMANI	Project leader												
SUHA BINTI IRWAN	Graph designer												
NURFATHIN AQILAH BINTI RASHIDE	Algorithm developer												
LIU MIAOYAN	Tester												
KRISSHA REDDY A/P APPLASWAMY REDDY	Documentation and presentation												